

AI-BASED RESUME SCREENING AND AI-ASSISTED INTERVIEW EVALUATION FRAMEWORK WITH PROCTORING SYSTEM

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Abstract – Automated resume screening ranks candidates on self-reported documents and offers no mechanism for revising that judgment when later evidence contradicts it. This paper investigates whether an adaptive artificial intelligence (AI) interview stage can correct the ranking errors introduced by resume-only screening, and what data exposure such a pipeline entails. We present a locally deployable hiring pipeline that integrates a deterministic applicant tracking system (ATS) keyword pre-filter, large language model (LLM) resume scoring, an adaptive interview, proctoring-based integrity scoring, and trust-weighted score fusion. A controlled audit is performed on a 100-candidate synthetic cohort with concealed competence values and planted over-sellers, hidden gems, and honest controls, while transport-level interception quantifies data egress. Across four resume-only rankers, over-sellers occupied six or seven of the top ten positions. Score fusion raised normalized discounted cumulative gain at rank ten (NDCG@10) from 0.61 to 0.94 and Kendall's tau-b from 0.17 to 0.55, and it recovered every hidden gem. However, a false correction rate of 0.62 indicates that the interview stage is high-recall but imprecise. Candidate answers appeared in 12 of 14 model calls, compared with two for resumes, and substituting a local model reduced the bytes leaving the host from 41,798 to zero. All ranking results are simulated; interviewer validity, fairness, and encryption at rest remain unmeasured. These findings support treating interviews as independent measurements rather than as supplements to screening.

Index Terms: Automated hiring, resume screening, large language models, adaptive interviewing, score fusion, data privacy, algorithmic auditing.

INTRODUCTION

Large employers rely on automated screening to reduce a large applicant pool to a shortlist that human reviewers

can feasibly read. Applicant tracking systems (ATS) have traditionally performed this reduction by matching keywords or applying predefined criteria to resume text [1]. Subsequent systems adopted semantic representations, first through word embeddings combined with stable matching [2] and later through context-aware transformer models for resume classification and resume-job matching [3]. More recently, large language model (LLM) agents have been used to summarize and grade resumes at scale [4], and multi-agent designs have added explanations to the resulting scores [5]. Because hiring decisions affect people's livelihoods, the correctness, robustness, and data handling of these systems matter well beyond engineering convenience.

Despite this progress, resume screening and interview assessment are still usually treated as separate filters. A resume-based score is computed once and gates later stages, yet nothing allows subsequent evidence to revise it. This limitation is consequential because a resume is a self-reported document: one that overstates competence and one that understates it will each be ranked incorrectly, and a screener that reads only that document cannot detect either error. Moreover, evaluations of screening systems typically report accuracy or efficiency for a single component. Li et al. report 73.3% competence-level accuracy and 79.2% resume-job matching accuracy, with an inter-annotator kappa of 61% [3]. Gan et al. report screening roughly elevenfold faster than manual review and an F1 score of 87.7% on resume-sentence classification [4]. Neither study examines whether a later stage corrects errors made by an earlier one. Fairness and robustness research likewise analyzes screening in isolation. Fabris et al. survey bias across the algorithmic hiring pipeline and characterize whether algorithmic hiring is less biased than human-driven alternatives as an open question [6]. Baxi et al. show that

Commented [A1]: Do not complete the Author name section until you submit the final camera-ready copy.

Important notice about copyright:

During the final paper submission process, you must complete the electronic copyright form. Your paper cannot be published in the Proceedings if you do not complete this step.

There are specific copyright notices for Government agencies. Please contact the Proceedings Co-chairs for the correct statement to include in your paper.

Delete this comment before submitting your paper for the first review.

prompt-injection text embedded in a resume can improve an applicant's LLM-assigned ranking under some conditions, and that this advantage weakens as the tactic becomes widespread [7]. That finding concerns screening alone, not a pipeline in which a subsequent interview could counteract an inflated score.

This paper addresses that gap. We investigate whether an adaptive artificial intelligence (AI) interview stage can correct ranking errors introduced by resume-only screening, and what data exposure such a pipeline entails. The contributions of this work are as follows:

1. We implement a locally deployable hiring pipeline comprising a deterministic keyword pre-filter, LLM resume scoring, an adaptive interview, proctoring-based integrity scoring, and trust-weighted score fusion. The fusion step allows an untrusted interview to stop benefiting a candidate without penalizing that candidate.
2. We design a controlled audit on a synthetic cohort with concealed competence values. The cohort includes planted over-sellers, hidden gems, and two honest control groups, so that genuine correction can be distinguished from noise.
3. We compare four resume-only rankers on the same cohort and report how many over-sellers each admits to the shortlist.
4. We quantify data egress by intercepting the transport layer, comparing a hosted model provider with a local one.
5. We release a measurement harness that labels every reported quantity by provenance and states explicitly which quantities could not be measured.

In the audit, over-sellers occupied six or seven of the top ten positions under all four resume-only rankers. Fusing the interview stage raised normalized discounted cumulative gain at rank ten ($\text{NDCG}@10$) from 0.61 to 0.94 and Kendall's tau-b from 0.17 to 0.55, and it recovered every planted hidden gem. However, 62% of the rank changes it made were not corrections of a planted error, so the stage is better characterized as high-recall than precise. Candidate answers appeared in 12 of 14 model calls, compared with two for resume text, and substituting a local model reduced the bytes leaving the host from 41,798 to zero. All ranking results are simulated; they evaluate the fusion function on a constructed cohort rather than the validity of the interviewer or real hiring outcomes.

The remainder of this paper is organized as follows. Section II reviews related work on resume screening, LLM-based evaluation, and fairness and robustness. Section III describes the system design. Section IV presents the evaluation methodology, including the synthetic cohort and provenance labels. Section V reports the results. Section VI discusses threats to validity,

Section VII addresses limitations and future work, and Section VIII concludes the paper.

LITERATURE SURVEY

Prior work relevant to a hiring pipeline in which an interview stage can revise a resume-based ranking falls into four themes: keyword and rule-based screening, semantic and embedding-based matching, LLM-based screening and evaluation, and fairness, bias, and robustness. This section assesses what each theme measures and what it leaves unexamined, and it concludes by stating the research gap addressed by this paper.

A. Keyword and Rule-Based Screening

Early applicant tracking systems treated screening as document scoring. Resumes were parsed into fields and compared with job requirements through largely deterministic logic [1]. This design is fast, and its criteria are explicit and reproducible. Its main weakness is that it cannot recognize that two differently worded descriptions refer to the same competency. The weakness is also structural: a keyword score measures the presence of resume terms rather than substantiated competencies, so a resume that lists every requirement scores well regardless of whether the candidate can support those claims.

B. Semantic and Embedding-Based Matching

Later work represented resumes and job descriptions in a shared vector space to close the semantic gap. Pudasaini et al. combined word embeddings with the Gale–Shapley stable-matching procedure, which recasts screening as a matching problem over the preferences of both sides rather than a one-directional ranking [2]. Vanetik and Kogan address job vacancy ranking by combining sentence embeddings, keywords, and named entities [8]. Li et al. used context-aware transformer models to condition a representation on its surrounding resume section and the paired job description. They evaluated on 6,492 annotated resumes for a single occupation and reported 73.3% competence-level accuracy and 79.2% resume–job matching accuracy. Their inter-annotator Kappa of 61% shows that expert judgment itself does not fully converge [3]. This indicates disagreement, and therefore uncertainty, in the human-labelled target. These methods capture relatedness better than literal matching, but embedding similarity measures how related two texts are, not whether the candidate can do the job. Better similarity therefore does not by itself show better hiring decisions. Like the keyword approach, these methods operate primarily on self-reported resume content.

C. LLM-Based Screening and Evaluation

The most recent systems replace fixed similarity with generative evaluation. Gan et al. describe an LLM-agent framework that summarizes and grades resumes, reporting

screening roughly eleven times faster than manual review and an F1 of 87.7% on resume-sentence classification after fine-tuning [4]. Lo et al. decompose screening across specialized agents and, in some configurations, ground scores in retrieved role-specific criteria to make outputs explainable [5]. Human and LLM-Based Resume Matching compares human ratings with LLM matching scores [9], which is directly relevant to validating a model against human judgment. These systems produce natural-language justifications alongside scores. That output is richer than a similarity value, but it introduces an additional evaluation requirement, since a fluent natural-language justification is not necessarily a faithful account of the process that produced the score. The reported evaluations concern efficiency, classification accuracy, or agreement for the screening step. In these evaluations, the model's input is primarily the resume.

D. Fairness, Bias, and Robustness

As these systems have scaled, so has scrutiny of their behavior. Fabris et al. survey fairness and bias across the algorithmic hiring pipeline, covering bias sources, mitigation techniques, and legal frameworks. They also state that whether algorithmic hiring is less biased than the human-driven alternatives remains an open question [6]. Empirical studies probe LLM matching directly. One evaluates the effect of gender, race, and education on job-resume matching [10], and FAIRE offers a benchmark-style test of how identity-related changes affect resume evaluation [11]. Robustness has been studied separately. Baxi et al. show that self-promotional prompt-injection text embedded in a resume can improve an applicant's LLM-assigned ranking when few applicants use it and quality is otherwise homogeneous. The advantage weakens as the tactic spreads, and in heterogeneous settings it can occasionally invert the ranking of weaker and stronger candidates [7]. This work matters because it shows that resumes can constitute an attack surface, not only an unreliable signal.

E. Critical Synthesis

Across these themes, capability has moved from literal matching to contextual and generative evaluation, but the evidence supplied is not directly comparable. The studies use different datasets, tasks, and metrics, from competence classification [3] to sentence classification [4] to injection-induced rank change [7]. The reviewed evaluations also share three limitations. First, the reviewed studies generally examine one component in isolation. Screening accuracy is reported separately from robustness, and neither is examined within a pipeline. Second, the reviewed screening approaches, from keyword scoring to LLM agents, primarily evaluate resume content, and none of the reviewed studies evaluates whether subsequent interview evidence can revise the resulting ranking. When that document overstates or understates a candidate, better

reading of it alone does not address the discrepancy. Third, data handling is not the focus of these studies. None of the sources reviewed measures transport-level data egress, that is, what candidate data leaves the deploying organization, which matters when hosted models process resumes and interview answers.

F. Research Gap and Position of This Work

In the sources reviewed, resume screening and interview assessment are evaluated as separate problems, and none tests whether a subsequent interview stage can revise rankings produced by an earlier resume-based stage. The robustness finding in [7] is reported for screening alone, not for a pipeline in which a subsequent interview could revise the resulting ranking. This paper addresses that gap. It treats screening and adaptive interviewing as stages of one auditable ranking process. A controlled audit plants over-sellers, hidden gems, and honest controls in a synthetic cohort with concealed competence values, so that rank revisions that reflect planted errors can be distinguished from rank noise. It also measures transport-level data egress for hosted and local model providers. Fairness auditing is outside the scope of this study, so the fairness literature [6], [10], [11] is discussed as contextual background rather than as evidence that the proposed system is fair.

PROPOSED SYSTEM

This section describes an end-to-end hiring pipeline built so that a later evidence source can revise an earlier ranking. The pipeline has five processing stages: keyword pre-filter (preceded by ingestion and sanitization), model screening, adaptive interview, integrity scoring, and trust-weighted fusion into a ranked report. Every decision is made in a service layer that is independent of the user interface, so the same code drives the web application and the measurement harness. The architecture is shown in Fig. 1 and the stage flow in Fig. 2. Once a resume has been ingested, a candidate can be removed automatically in only one place, the keyword pre-filter. A low model score routes the candidate to human review, not to rejection.

A. System Architecture and Design Decisions

The system is a Python application with four kinds of module. Pages render, services decide, core modules hold pure scoring code, and a provider layer is the only path by which data can reach a model. State is stored in SQLite (11 tables, write-ahead logging, enforced foreign keys with cascading deletes). Fig. 1 marks a host trust boundary. The application runs as a single process on the operator's machine, and the provider factory is the single point at which a request can leave the host. Whether a given request crosses the boundary depends on the configured provider endpoint.

Four design decisions shape the rest of the section.

Provider abstraction. Four provider types are supported: a hosted Gemini API, a local Ollama daemon, an OpenAI-compatible endpoint, and an offline fake used for testing. The first three are reached over plain REST, and the fake makes no network request. The provider is selected through configuration, and the same prompts are built in every case.

Deterministic first. Sanitization, keyword scoring, integrity scoring, fusion, and ranking make no model call. A throttled provider can therefore delay a decision but not lose a submission.

Recomputed totals. The model supplies five criterion scores and prose. Totals are recomputed from the clamped criteria, so a model-reported total cannot change an outcome.

Prompts in files. Five prompt templates are stored as text files and can be inspected without reading source code.

B. Stage 1: Ingestion, Sanitization, and Keyword Pre-filter

Resume text is extracted with PyMuPDF (PDF), python-docx (DOCX), or direct read (text). It is then normalized. Text below a 40-word floor is refused as unparseable and is not scored. No structured field extraction (skills, roles, years) is performed. The cleaned text is passed to the model in full, truncated at 18,000 characters. This avoids adding a parser as a second error source that could not be separated from screening error.

Sanitization strips instruction-like text using five regular-expression families: instruction override, persona hijack, chat markup, score demand, and verdict demand. It never rejects a resume. It removes the matched text and records a flag on the application, so a recruiter sees what was changed. This conservative handling is deliberate, because a pattern match cannot separate an attack from a security engineer describing their work.

The pre-filter score is the share of a role's listed requirements found literally in the resume, using word-boundary matching:

$$SS_{\text{ats}} = \begin{cases} \operatorname{round}\left(\frac{\operatorname{count}(\text{matched})}{\operatorname{count}(\text{requirements})}\right), & \text{if } \operatorname{count}(\text{requirements}) \geq 1 \\ 0, & \text{otherwise} \end{cases}$$

When a role lists no requirements, the ratio is undefined. By convention SS_{ats} is then set to 100, so an empty criterion set cannot cause a rejection. A resume with $SS_{\text{ats}} < \tau_{\text{ats}} = 40$ is rejected without a model call. The rejection reason names the missing requirements. The filter is intentionally lexical, so every automatic rejection can be explained to the candidate as a list of terms.

C. Stage 2: Model Screening

For resumes that pass the pre-filter, one schema-constrained model call scores five criteria from 0 to 5: skill match, experience, projects, communication, and culture

fit. The prompt marks the resume as untrusted data. It also lists the requirements that the literal search did not match, so the model can credit genuine equivalents. The screening score is the sum of the clamped criteria, giving $SS_{\text{resume}} \in [0, 25]$.

This score gates entry to the interview. A candidate with $SS_{\text{resume}} \geq s_{\min}$, where $s_{\min}$ is the shortlist floor, is shortlisted and proceeds to the interview stage. A candidate with $SS_{\text{resume}} < s_{\min}$ is routed to human review. The pipeline neither rejects nor interviews that candidate automatically.

D. Stage 3: Adaptive Interview

On shortlisting, a question plan of at most 12 questions is generated from the resume and job description. Each question is tagged by source: resume, job-description gap, culture, or behavioral. Requirements for which the resume provides no evidence are converted into job-description-gap questions. The interview therefore probes claims the document does not substantiate and gives the candidate an opportunity to demonstrate competence that the resume does not show.

After each answer, a turn policy chooses one of four actions: probe, steer, next planned, or wrap up. Bounds are checked before the model is consulted. The turn budget, the wall clock, and the unasked plan are enforced first, and at most two adaptive turns may occur in a row. Within the turn budget and wall clock, the model cannot end the interview while planned questions remain, and a planned question is asked in its exact wording. A steer is triggered by a keyword match against watch topics, so the decision to follow the candidate needs no model call.

Each answer passes a two-tier guardrail. Tier 1 is structural and pattern-based. Tier 2, a model check, runs only when word overlap with the question falls below 0.06, and it fails open with a flag if the provider is unavailable. A rejected answer returns the question without spending the turn. After three rejections the answer is accepted with a forced accept flag, so a guardrail cannot lock a candidate out. At the end, one call scores five criteria (technical depth, problem solving, communication, culture fit, practical impact), each from 0 to 5. The total $SS_{\text{interview}} \in [0, 25]$ is again recomputed from the clamped criteria. The deadline is the earlier of the session duration and the window close, and a refresh resumes the interview without spending the single attempt.

E. Stage 4: Integrity Scoring

Proctoring runs in-process alongside the interview. It combines computer vision (face detection, head pose, identity matching, and optional object detection), audio loudness cues, and browser signals. Its output is a set of discrete events SE . Each event has one of 13 kinds and one of four severities, whose weights w_{s_i} are 12

(critical), 6 (high), 3 (medium), and 1 (low). The integrity score is computed as

$$d = \frac{\sum_{k \in K} w_{s_k} \cdot \frac{1}{1 + \gamma \ln |E_k|}}{\sum_{k \in K} w_{s_k} \cdot \frac{1}{1 + \gamma \ln |E_k|}}$$

with $\gamma = 1$, $\sigma = 0.25$, and $\kappa = 2$. Here K is the set of distinct kinds observed, E_k is the set of events of kind k , and s_k is the worst severity observed for kind k . Each distinct kind is thus charged the weight of its most severe event, multiplied by $1 + \gamma \ln |E_k|$, which grows logarithmically with the number of events of that kind, so repetition raises the charge sublinearly. The sum over kinds is then multiplied by a factor that increases with the number of distinct kinds and is capped at κ . The quantity d is the total deduction, and E is the integrity score on a 0–100 scale.

The verdict is flag below 55, clean at 80 or above with no critical event, and review otherwise. Any critical event precludes a clean verdict. A single critical event, which by the arithmetic alone leaves a score of 88, is therefore held at review. The function is deterministic and order-independent, so it can be recomputed from stored events without the video. It reports evidence for a human reviewer and is not an automated finding of misconduct.

F. Stage 5: Trust-Weighted Fusion and Ranking

Integrity does not gate or multiply the final score. It determines how much weight the interview receives. Let the trust be $t = c(E)/100$, let $\lambda \in [0, 1]$ be the interview weight (default 0.5), and note that S_{resume} and $S_{\text{interview}}$ are both expressed on the 0–25 rubric scale. Then

$$v = t \cdot S_{\text{interview}} + (1 - t) \cdot S_{\text{resume}}, \quad S_{\text{final}} = (1 - \lambda) \cdot S_{\text{resume}} + \lambda \cdot v$$

Equivalently,

$$S_{\text{final}} = S_{\text{resume}} + \lambda \cdot t \cdot (S_{\text{interview}} - S_{\text{resume}})$$

so the effective interview weight is $\lambda \cdot t$. As $t \rightarrow 0$, S_{final} collapses onto S_{resume} , not onto zero. Low integrity therefore removes the interview's influence but adds no penalty of its own. When trust is high, interview performance can move S_{final} above or below S_{resume} . Candidates are ordered by $(-S_{\text{final}}, -S_{\text{resume}}, -S_{\text{ats}}, \text{application id})$. The final key, the application id, makes the ordering total and deterministic, which is required because the 26-value integer rubric produces many ties.

Algorithm 1. End-to-end candidate evaluation

Input: resume file F , role J (with requirement list), floor τ_{ats} , shortlist floor s_{min} , interview weight λ

Output: outcome, and S_{final} for interviewed candidates

```
1: R ← normalize(extract(F))
2: if |words(R)| < 40 then return REFUSED(unparseable)
3: R' ← sanitize(R)           ▷ strip and flag, never reject
4: S_ats ← keyword_score(R', J)   ▷ 100 if J lists no requirements
5: if S_ats < τ_ats then return REJECTED(missing terms)
6: S_res ← Σ clamp(model_criteria(R', J), 0, 5) ▷ five criteria
7: if S_res < s_min then return UNDER_REVIEW ▷ human review, no interview
8: plan ← build_plan(R', J); E ← ∅; transcript ← ∅
9: while turn budget and clock allow:
10:  q ← next_turn(plan, transcript)   ▷ probe, steer, next planned, or wrap up
11:  if q = WRAP_UP then break
12:  a ← accept_answer(q)             ▷ ≤3 rejections, then forced_accept
13:  transcript ← transcript ∪ {(q, a)}
14:  E ← E ∪ proctor_events()
15:  S_int ← Σ clamp(model_interview_criteria(transcript), 0, 5)
16: t ← c(E)/100; v ← t · S_int + (1 - t) · S_res
17: S_final ← (1 - λ) · S_res + λ · v
18: return S_final                 ▷ candidates then ordered by the ranking key
```

G. Implementation

The system is written in Python 3.12 with Streamlit 1.63 for the interface and 17 pinned, free and open-source dependencies. Model calls use plain REST through requests, with no vendor SDK. Speech recognition uses faster-whisper, and vision uses OpenCV (YuNet and SFace) and MediaPipe. Object detection uses YOLO11n through ultralytics, which is AGPL-3.0 licensed and disabled by default. Recorded media, where retained, stays on the operator's local disk and is not written to the database, which stores derived event records. The test suite collects 371 cases and runs offline with the fake provider. Table 1 summarizes the modules.

TABLE 1
MODULES, INPUTS, OUTPUTS, AND TECHNIQUES

Module	Input	Output	Technique	Model call
Sanitizer	resume text	cleaned text, flags	5 regex families	none
Pre-filter	text, requirements	S_ats, missing list	literal word-	none

			boundary match	
Screening	text, role	$S_{\text{resume}}(0-25)$	5-criterion rubric, schema-constrained	1
Interview	plan, answers	transcript, $S_{\text{interview}}$	bounded turn policy, 2-tier guardrail	plan, turns, guardrail (tier 2), score
Integrity	events	$c(E)$, verdict	per-kind worst severity, log persistence	none
Fusion	three scores	S_{final} , rank	trust-weighted convex mix	none

H. Link to Evaluation

Section IV evaluates the pipeline on a 100-candidate synthetic cohort generated from a fixed seed. Concealed competence values are assigned, and over-sellers, hidden gems, and two honest control groups are planted by construction. Resume-only rankers (keyword, TF-IDF, BM25, and a simulated model score) are compared with the fused ranking. Ranking metrics, control-case rank displacement, and transport-level data egress under hosted and local providers are also reported.

RESULT

This section reports measurements obtained from four independent evaluation sources: a 100-candidate synthetic cohort (seed 20260923, with 25 candidates in each of four planted cases — over-sellers, hidden gems, honest-strong controls, and honest-weak controls — with competence values concealed from all rankers), used for the ranking and fusion audit; 26 hand-labelled boundary cases, used to characterize the keyword pre-filter; 31 attack strings and 18 benign strings, used to characterize the prompt-injection guardrail; and offline benchmarks of the implemented codebase (latency, data egress, provider contract, and implementation inventory), executed with the fake provider except where a live call is explicitly stated. Ranking and fusion quantities in this section are computed on the synthetic cohort described above and are simulated; they are not claims about real candidates. Quantities requiring a local model run, human raters, demographic labels for a fairness assessment, or labelled video or audio media were not measured and are not reported.

A. Comparison of Resume-Only Rankers

Four resume-only rankers were scored against concealed competence on the synthetic cohort (Table II, Fig. 5). BM25 reached an NDCG@10 of 0.6234, followed by TF-IDF (0.5846), the simulated model score (0.5572), and the keyword score (0.4843). Over-sellers occupied seven of the top ten positions under the keyword score and TF-IDF, and six under BM25 and the simulated model score. The fused ranking (Section V-B) reached an NDCG@10 of 0.9440 on the same cohort and is included in Table II for reference.

TABLE II. Resume-Only Rankers on the Simulated Cohort

Ranker	NDCG@10	Over-sellers in top 10
Keyword (ATS)	0.4843	7
TF-IDF (cosine)	0.5846	7
BM25	0.6234	6
Simulated model score	0.5572	6
Fused (for reference)	0.9440	—

Precision@10 and mean reciprocal rank varied widely across re-drawn cohorts (Section V-B) and are not reported here as point estimates.

B. Rank Correction by Fusion

On the single shipped cohort draw described above, fusion raised NDCG@10 from 0.6087 to 0.9440 and Kendall's tau-b from 0.1701 to 0.5471 (Fig. 6). All planted hidden gems moved up in rank (recovery rate 1.00), with a mean gain of 21.4 positions. Over-sellers fell by a mean of 24.48 positions. The control displacement rate — the proportion of honest controls (honest-strong and honest-weak) displaced by more than five positions — was 0.62.

To assess whether these figures depend on the specific cohort draw, the cohort was regenerated 200 times using seed 20260923 and its 199 successive integer seeds, and both the resume-only and fused rankings were recomputed on each regeneration. This 200-draw set is distinct from, and does not replace, the single shipped-cohort figures reported above: it characterizes the variability of those figures rather than their point value. Across the 200 regenerations, the mean tau-b of the fused ranking was 0.5812, the mean tau-b gain over resume-only ranking was 0.3826, and the mean NDCG@10 gain was 0.2855.

The interview-weight sweep from 0 to 1 in steps of 0.05, run on the single shipped cohort, is shown in Fig. 7. Tau-b peaked at 0.790 at a weight of 0.95, against 0.547 at the shipped weight of 0.5. When the same sweep was repeated on each of the 200 cohort regenerations, the maximizing weight did not fall below 0.8 in any regeneration and was not identical across regenerations. The shipped weight of 0.5 was fixed before this sweep was run and was not retuned in light of it.

C. Integrity Scoring

An event-free session scored 100 (clean), and a single critical event scored 88 (review) (Fig. 8). Six repeated no-face events scored 83 (clean); forty repeated no-face events scored 72 (review). A session with a phone, written notes, a background voice, and two glances away scored 75 (review). The score was non-increasing over 39 cumulative event additions, identical over 1,000 repeated calls on a fixed input, and identical across every ordering of a fixed four-event set.

D. Prompt-Injection Guardrail

The tier-1 guardrail was evaluated on 31 attack strings across five pattern families and 18 benign strings, 10 of which were hard negatives (Fig. 9). It flagged 28 of 31 attacks, an attack detection rate of 0.9032. This rate reflects detection of the specific attack strings tested and is not reported as a general resistance guarantee against unseen attacks. The three evasions were a leetspeak variant, a hyphenated variant, and a polite paraphrase with no imperative verb. Four benign strings were flagged, all hard negatives, giving a false-positive rate of 0.2222 overall and 0.40 on hard negatives. Precision was 0.875, recall 0.9032, F1 0.8889, specificity 0.7778, and accuracy 0.8571. The tier-2 escalation rate on benign input was 0.7778.

Sanitization does not reject a hostile resume outright: matched instruction-like text is removed from the resume, the removal is recorded as a flag on the application, and the application proceeds through the remaining screening steps. No hostile resume among the 31 tested was rejected by sanitization. All 28 resumes whose text was rewritten carried their corresponding flags, and no resume was flagged without also being rewritten.

E. Keyword Pre-Filter

On 26 hand-labelled boundary cases, the pre-filter matched the assigned label in 24 (accuracy 0.9231). The two misclassifications were both token-boundary errors rather than semantic gaps: a word-boundary check failed to exclude the single-character requirement "C" from matching inside "C++," and the requirement "SQL" was matched as an accidental substring of "PostgreSQL." Three further cases in the corpus probed synonym, abbreviation, and conceptual equivalence (for example, "relational databases" for "SQL," "ML" for "machine learning," and "Infrastructure as Code" for "Terraform"); for a literal keyword matcher, the corpus defines the correct answer in each of these three cases as no match, and the pre-filter returned no match in all three, so all three were scored correctly. Restricted to the 23 cases decidable from literal character content alone — that is, excluding the three semantic-equivalence probes above — accuracy was 0.9130 (21 of 23), consistent with both of the two actual misclassifications falling inside this subset. A five-requirement role admits six score values, and the threshold of 40 corresponds to at least two matched requirements.

F. Latency

Offline per-stage latency was measured with the fake provider; live model inference latency is excluded from every figure in this subsection (Table III, Fig. 10). Each stage's timing was recorded over 40 trials, except PDF extraction (10), script operations (8), candidate registration (6), and the end-to-end service stages (15). Script hashing and verification, at 118.31 ms and 118.99 ms at p50, and registration, at 119.98 ms, were the slowest stages. The scoring stages (keyword pre-filter, sanitization, integrity scoring, ranking) ran below 0.5 ms at p50. Ranking 100 candidates took 0.378 ms at p50.

Separately from these timing trials, four deterministic scoring functions — the keyword pre-filter, sanitization, integrity scoring, and ranking — were each called 50 times on a fixed input and returned identical output on every call. This determinism check does not extend to every offline stage: password hashing, for example, is salted by design and is expected to return different output on each call, so its timing is reported without a corresponding determinism claim.

TABLE III. Offline Stage Latency (ms), Fake Provider

Stage	p50	p95
PDF text extraction (9 pages)	49.11	49.55
Keyword score	0.075	0.076
Sanitization (hostile input)	0.102	0.147
Integrity score, 10 events	0.0048	0.0049
Integrity score, 200 events	0.0668	0.0674
Rank 100 candidates	0.378	0.414
Script hash	118.31	120.31
Register candidate	119.98	120.81
Apply (no model call)	0.441	0.569
Screen (fake model)	0.481	0.549

G. Data Egress

Transport-level interception recorded every outbound request body issued during one complete candidate journey of 14 model calls under each of the hosted and local provider configurations (Table IV, Fig. 11). The hosted provider sent 41,798 bytes to one external, non-loopback host; the local provider sent 40,870 bytes to a loopback address, so that its traffic was captured but did not leave the host. The fake provider issued no requests. The 0.9778 ratio between the local and hosted payload sizes is reported only as a comparison of request size; it does not, by itself, indicate anything about where the data was sent or how it was protected, which are reported separately above.

Resume text appeared in 2 of the 14 calls, and candidate answers appeared in 12 of 14. Comparing the two resume sizes used in this measurement (546 and 5,469 bytes), the request-body size increased at a measured slope of 2.0 request bytes per resume byte, with a fixed overhead of 39,778 bytes per candidate independent of resume length; both figures are derived from this two-point comparison rather than from a larger sweep of resume sizes.

The 57 intercepted request bodies referenced below aggregate every request captured across the hosted, local, and stub provider configurations over the full egress measurement run, including both the standard 14-call journey summarized in Table IV and the additional resume-size variants used to measure the slope above; a full per-configuration reconciliation of this count was not included in the supplied measurement data. No media file path, media file extension, data URI, or base64-encoded blob appeared in any of the 57 intercepted request bodies.

TABLE IV. Egress per Candidate Journey

Provider	Requests	Request bytes	Non-loopback egress	Bytes crossing network
Gemini (hosted)	14	41798	Yes	41798
Ollama (local)	14	40870	No (loopback only)	0
Fake	0	0	No	0

H. Provider Contract and Implementation Inventory

Provider contract. Two schemas were shipped to the hosted provider, using 11 allowed keywords. Sanitization preserved every required field and dropped no keyword the schemas use. In a live run, both shipped schemas were accepted, and both the pre-fix schema and an unsanitized schema were rejected. The mean hosted round trip was 4.146 s. One observed live call billed 603 reasoning tokens; this figure describes that single observed call and is not reported as a representative or average statistic for reasoning-token usage. The hosted response carried no generation-time field. Five model call sites exist in the implementation, of which two are schema-constrained.

Implementation inventory. The source comprises 33 files with 10,423 lines (8,361 code lines). The test suite has 15 files, 358 test definitions, and 371 collected cases, a test-to-source code-line ratio of 0.428. The database has 11 tables, 121 columns, 7 foreign keys with 7 cascading deletes, 22 personal-data columns, and 4 biometric-media columns. The system defines 13 proctoring event kinds and 5 guardrail pattern families, and has 17 pinned dependencies.

CONCLUSION

Resume-only screening ranks candidates on a self-reported document and cannot revise its judgment when later evidence contradicts it. This work asked whether an adaptive interview stage can revise the resulting ranking errors, and what such a pipeline exposes about candidate data. We built a locally deployable pipeline in which the interview is treated as an additional measurement source, combined with the resume score through trust-weighted

fusion, so that low-integrity interview evidence contributes less to the final score without lowering the candidate's resume score. We then audited it on a synthetic cohort with concealed competence values and planted errors.

On that cohort, four resume-only rankers let six or seven over-sellers into the top ten. Fusion raised NDCG@10 from 0.61 to 0.94 and Kendall's tau-b from 0.17 to 0.55, and every planted hidden gem moved up in rank. The fusion stage also displaced 62% of honest controls by more than five positions, so its corrections come with a measurable cost. At the transport layer, candidate answers appeared in 12 of 14 model calls, against 2 for resume text. Using a local model reduced measured non-loopback egress from 41,798 bytes to zero.

These ranking results are simulated. They characterize what the fusion function does with interview evidence of a stated quality; they do not establish that the interviewer supplies evidence of that quality, and they make no claim about outcomes on real hiring data. Future work should score recorded interviews with at least three human raters to assess agreement with the interviewer's output, run the ablation ladder and a real zero-shot baseline with a local model, add encryption at rest and scheduled retention, and extend the synthetic generator to support a disaggregated fairness study.

FUTURE SCOPE

This study showed that fusing an interview stage into resume-only ranking revises planted errors on a synthetic cohort, but did not establish that a real interviewer supplies evidence of the assumed quality. Closing that gap, and the related gaps below, requires five extensions.

First, interviews conducted by a local model should be recorded and scored independently by at least three human raters, with inter-rater agreement and model-to-human correlation reported, to replace the assumed interview-noise model with a measured one.

Second, the planned ablation ladder should be executed with a real zero-shot language-model baseline and an embedding-based baseline, both run against a local model, so that each stage's contribution to the observed correction can be attributed rather than assumed.

Third, local deployment should be characterized by inference latency, throughput, and peak memory, reported alongside the specific model, quantization, and hardware used, since none of these was measured in the present study.

Fourth, encryption at rest and a scheduled data-retention job should be implemented to close the data-protection gaps identified here; until then, the local-deployment egress result characterizes network exposure only, not storage-level protection.

Fifth, the synthetic generator should be extended to encode protected attributes, enabling a disaggregated fairness study, and the evaluation should be repeated on both

technical and non-technical roles and on real resumes obtained under a documented licence. Each of these is offered as a precondition for stronger evidence, not as a claim already supported by this work. The present results are confined to a constructed synthetic audit; extending them along the five directions above is necessary before any claim of readiness for supervised use on real candidate data would be warranted, including in

organizations that process such data on their own hardware.

REFERENCES